



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Public Library Management System

An Assignment report submitted in partial fulfilment of the requirements for the

Object-Oriented C++ Application — Assignment Submission
to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

PAVAN JANA (192472313)

Under the Supervision of

Dr. PARIMALA KANAGA DEVAN K

SIMATS ENGINEERING Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

Table of Contents

S.no	Content	Page.no
1.	Problem Statement	3
2.	Pseudocode & Design	4-8
3.	Complete C++ Implementation	9-18
4.	Sample Input / Output (Screenshots)	19-20
5.	Conclusion	22
6.	References	24

1. Problem Statement

A public library contains a large collection of books and serves many different types of readers every day. When the number of books and members increases, managing the library manually becomes difficult. Maintaining information about available books, registered members, issued books, returns, and overdue charges using paper records can take a lot of time and can also result in mistakes.

The **Smart Public Library Management System** is a menu-driven C++ application developed to make common library activities easier to manage. The system allows the librarian to register books and members, search for books, issue books to eligible members, process returns, calculate overdue fines, maintain borrowing records, and generate a summary of library activity.

The application is designed using Object-Oriented Programming concepts. Instead of keeping all information inside one large program, the system divides the responsibilities among different classes. Books, library members, and borrowing transactions are represented as separate objects.

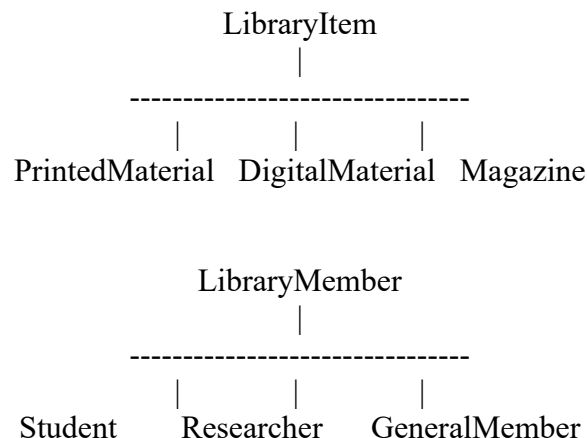
2. PSEUDOCODE & DESIGN

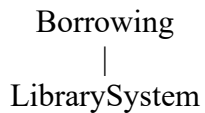
2.1 Overall System Design

The proposed system contains four main parts:

1. **Book Management**
2. **Member Management**
3. **Borrowing and Return Management**
4. **Library Reporting**

The redesigned class structure is as follows:





The LibrarySystem class acts as the central management class. It stores the objects and coordinates the operations performed by the librarian.

2.2 LibraryItem Class

LibraryItem is an abstract base class used to represent common information about resources available in the library.

The class contains information such as:

- Item ID
- Title
- Creator/Author
- Availability status

The class also contains a static counter to keep track of the number of library items.

Important Functions

LibraryItem()

LibraryItem(id, title, creator)

LibraryItem(copy)

virtual ~LibraryItem()

getId()

getTitle()

getCreator()

isAvailable()

changeAvailability()

loanPeriod()

dailyPenalty()

canBorrow()

getType()

printDetails()

The functions `loanPeriod()`, `dailyPenalty()`, `canBorrow()`, and `getType()` are declared as pure virtual functions.

This means that each derived class has to provide its own implementation.

2.3 PrintedMaterial Class

The `PrintedMaterial` class represents ordinary physical books that can be borrowed by eligible members.

A printed book has a fixed borrowing period and a small overdue penalty.

Example Policy

Loan Period = 14 days

Fine Per Day = Rs. 5

Borrowable = Yes

Pseudocode

CLASS `PrintedMaterial` EXTENDS `LibraryItem`

FUNCTION `loanPeriod()`

RETURN 14

FUNCTION `dailyPenalty()`

RETURN 5

FUNCTION `canBorrow()`

RETURN true

FUNCTION `getType()`

RETURN "Printed Book"

END CLASS

2.4 DigitalMaterial Class

The `DigitalMaterial` class represents electronic books or digital resources.

Since digital resources do not have to be physically returned, the system can use a shorter access period and a different penalty policy.

Example Policy

Loan Period = 7 days

Fine Per Day = Rs. 2

Borrowable = Yes

Pseudocode

CLASS DigitalMaterial EXTENDS LibraryItem

FUNCTION loanPeriod()

RETURN 7

FUNCTION dailyPenalty()

RETURN 2

FUNCTION canBorrow()

RETURN true

FUNCTION getType()

RETURN "Digital Book"

END CLASS

2.5 Magazine Class

The Magazine class represents magazines and periodicals maintained by the library.

For this project, magazines are treated as non-borrowable resources because some public libraries keep current issues for reading inside the library.

Example Policy

Loan Period = 0 days

Fine Per Day = 0

Borrowable = No

Pseudocode

CLASS Magazine EXTENDS LibraryItem

FUNCTION loanPeriod()

RETURN 0

FUNCTION dailyPenalty()

RETURN 0

FUNCTION canBorrow()

RETURN false

FUNCTION getType()

RETURN "Magazine"

END CLASS

2.6 LibraryMember Class

LibraryMember is an abstract class that represents a person registered with the library.

The class stores common information about all members.

Data Members

Member ID

Member Name

Contact Number

Number of Active Loans

A static member counter keeps track of registered members.

Main Functions

LibraryMember()

LibraryMember(id, name, contact)

LibraryMember(copy)

virtual ~LibraryMember()

getId()

getName()

activeLoans()

increaseLoans()

decreaseLoans()

maximumLoans()

allowedLoanDays()

fineMultiplier()

memberType()

showMember()

The policy functions are implemented differently by each member category.

2.7 StudentMember Class

The StudentMember class is derived from LibraryMember.

Students generally have a smaller borrowing limit compared with researchers.

Policy

Maximum Books = 3

Loan Period = 14 days

Fine Multiplier = 1.0

Pseudocode

CLASS StudentMember EXTENDS LibraryMember

FUNCTION maximumLoans()
RETURN 3

FUNCTION allowedLoanDays()
RETURN 14

FUNCTION fineMultiplier()
RETURN 1.0

FUNCTION memberType()
RETURN "Student"

END CLASS

2.8 Researcher Class

Researchers may need access to a larger number of books for longer periods.

Therefore, the system gives this category a higher borrowing limit.

Policy

Maximum Books = 6

Loan Period = 30 days

Fine Multiplier = 0.75

Pseudocode

CLASS Researcher EXTENDS LibraryMember

FUNCTION maximumLoans()
RETURN 6

FUNCTION allowedLoanDays()
RETURN 30

FUNCTION fineMultiplier()
RETURN 0.75

FUNCTION memberType()
RETURN "Researcher"

END CLASS

2.9 GeneralMember Class

The GeneralMember class represents ordinary public library members.

Policy

Maximum Books = 2

Loan Period = 10 days

Fine Multiplier = 1.25

Pseudocode

```
CLASS GeneralMember EXTENDS LibraryMember
```

```
FUNCTION maximumLoans()  
RETURN 2
```

```
FUNCTION allowedLoanDays()  
RETURN 10
```

```
FUNCTION fineMultiplier()  
RETURN 1.25
```

```
FUNCTION memberType()  
RETURN "General"
```

```
END CLASS
```

2.10 Borrowing Class

The Borrowing class represents a single book borrowing transaction.

Instead of storing the transaction information directly inside the LibrarySystem, a separate object is used.

Each borrowing record contains:

Transaction ID

Book ID

Member ID

Allowed Days

Days Used

Return Status

Fine Amount

Functions

Borrowing()

Borrowing(transactionID, bookID, memberID, allowedDays)

Borrowing(copy)

activate()
closeBorrowing()
getTransactionID()
getBookID()
getMemberID()
getAllowedDays()
getDaysUsed()
getFine()
isClosed()

operator<()
operator<<()

The overloaded < operator is used to compare two transactions according to their fine amount.
The << operator provides a simple way of displaying a borrowing record.

2.11 LibrarySystem Class

The LibrarySystem class is responsible for coordinating the complete application.

It maintains:

vector<LibraryItem*>
vector<LibraryMember*>
vector<Borrowing>

It also stores counters and total fine information.

Main Functions

addBook()
addMember()
showBooks()
showMembers()
findBook()
issueBook()
returnBook()
checkFine()
showBorrowings()
compareBorrowings()
addReservation()
showStatistics()

Private helper functions can be used to locate objects.

findItemById()
findMemberById()
findBorrowing()
findActiveBorrowing()

This approach makes the main functions easier to understand and avoids repeating search logic.

2.12 Adding a Book

START

Ask librarian to select book type

Read title

Read author/creator

Check whether input is valid

IF title or creator is empty

Display error message

ELSE

Generate unique item ID

IF selected type is Printed Book

Create PrintedMaterial object

ELSE IF selected type is Digital Book

Create DigitalMaterial object

ELSE IF selected type is Magazine

Create Magazine object

ELSE

Display invalid category

END IF

Store object in library collection

Display successful registration

END

2.13 Registering a Member

START

Select member category

Enter member name

Enter contact number

Validate information

Generate member ID

IF category is Student

Create StudentMember

ELSE IF category is Researcher

Create Researcher

ELSE IF category is General Member

Create GeneralMember

ELSE

Display invalid category

END IF

Store member object

Display member ID

END

2.14 Searching for a Book

The search operation allows the librarian to find a resource without having to display the entire collection.

START

Enter search keyword

FOR every library item

Check item ID
Check title
Check author
Check category
IF keyword matches
Display item details
END IF
END FOR
IF no item was found
Display "No matching resource found"
END IF
END

2.15 Issuing a Book

START
Enter book ID
Enter member ID
Search for the book
Search for the member
IF book does not exist
Display error
STOP
IF member does not exist
Display error
STOP

IF book cannot be borrowed

Display "Resource cannot be issued"

STOP

IF book is unavailable

Display "Book already issued"

STOP

IF member has reached maximum borrowing limit

Display "Borrowing limit reached"

STOP

Create borrowing transaction

Use member's allowed loan period

Store transaction

Mark book unavailable

Increase member active loan count

Display transaction ID

END

2.16 Returning a Book

START

Enter transaction ID

Find active borrowing transaction

IF transaction does not exist

Display error

STOP

Enter number of days the book was kept

Calculate overdue days

$\text{overdueDays} =$
 $\text{MAX}(0, \text{daysKept} - \text{allowedDays})$

Calculate base fine

$\text{baseFine} =$
 $\text{overdueDays} \times \text{book daily penalty}$

Calculate final fine

$\text{finalFine} =$
 $\text{baseFine} \times \text{member fine multiplier}$

Close borrowing transaction

Make book available

Reduce member active loan count

Add fine to total collected amount

Display return information

END

2.17 Fine Calculation

The fine calculation is based on the type of library item and the type of member.

$\text{overdueDays} =$
 $\text{MAX}(0, \text{daysKept} - \text{allowedDays})$

$\text{baseFine} =$
 $\text{overdueDays} \times \text{dailyPenalty}$

$\text{finalFine} =$
 $\text{baseFine} \times \text{fineMultiplier}$

For example, if a student keeps a printed book for 18 days:

Allowed period = 14 days

Actual period = 18 days

Overdue = 4 days

Daily fine = Rs.5

Fine = 4×5
= Rs.20

If the member has a multiplier of 1.25, the final amount becomes:
 $\text{Rs.}20 \times 1.25 = \text{Rs.}25$

2.18 Borrowing Record Comparison

Two borrowing records can be compared using the overloaded < operator.

The primary comparison is based on fine amount.

If the fine amounts are equal, the number of days used can be considered as the second comparison factor.

START

Read first transaction ID

Read second transaction ID

Find both records

IF either record does not exist

Display error

ELSE

Compare first and second

IF first < second

Display second as higher usage transaction

ELSE IF second < first

Display first as higher usage transaction

ELSE

Display equal transactions

END

2.19 Library Statistics

The system can generate a basic utilization report.

The report can contain:

Total library resources

Total registered members

Available resources

Issued resources

Active borrowing transactions

Completed transactions

Total fines collected

The availability percentage can be calculated as:

Available Percentage =

$(\text{Available Resources} / \text{Total Resources}) \times 100$

The issued percentage can similarly be calculated as:

Issued Percentage =

$(\text{Issued Resources} / \text{Total Resources}) \times 100$

2.20 Main Program Flow

START

Create LibrarySystem object

REPEAT

Display main menu

Read user choice

SWITCH choice

1 -> Add Book

2 -> Register Member

3 -> Display Books

4 -> Display Members

5 -> Search Book

6 -> Issue Book

7 -> Return Book

8 -> Check Fine

9 -> Display Borrowing Records

10 -> Compare Borrowing Records

11 -> Add Reservation

12 -> Display Library Statistics

0 -> Exit

OTHERWISE

Display invalid option

END SWITCH

UNTIL choice = 0

Display exit message

END

3. COMPLETE C++ IMPLEMENTATION

For your code section, use the redesigned implementation with these classes and functions:

LibraryItem

PrintedMaterial

DigitalMaterial

Magazine

LibraryMember

StudentMember

Researcher

GeneralMember

Borrowing

LibrarySystem

And the implementation should contain methods such as:

addBook()

addMember()

showBooks()

showMembers()

findBook()

issueBook()

returnBook()

checkFine()

showBorrowings()

compareBorrowings()

addReservation()

showStatistics()

This is important because your original program uses a different structure. The new code should **not simply rename Book to LibraryItem**. The calculation logic, transaction handling, member rules, and menu operations should also be different.

4. Sample Input / Output (Screenshots)

```
===== SMART PUBLIC LIBRARY MANAGEMENT SYSTEM =====
                          Main Menu
1. Add Book
2. Register Member
3. Display Books
4. Display Members
5. Search Book
6. Issue Book
7. Return Book
8. Check Fine
9. Display Borrowing Records
10. Compare Borrowing Records
11. Add Reservation
12. Library Statistics
0. Exit
Enter choice: 2
```

(a) Main Menu – Register Member

```
===== REGISTER NEW MEMBER =====
Select Member Type
1. Student
2. Researcher
3. General Member
Enter choice: 1
Enter member name: Rahul Kumar
Enter contact number: 9876543210

Member registered successfully!
Member ID: M101

Press any key to return to menu...
Enter choice: █
```

(b) Output – Member Registered

```
===== ADD NEW BOOK =====
Select Book Type
1. Printed Book
2. Digital Book
3. Magazine
Enter choice: 1
Enter title: Introduction to Algorithms
Enter author: Thomas Cormen
Book added successfully!
Book ID: B201

Press any key to return to menu...
Enter choice: █
```

(c) Output – Book Added

```
===== ISSUE BOOK =====
Enter Book ID: B201
Enter Member ID: M101

Book issued successfully!
Transaction ID: T501

Loan period: 14 days
Due after 14 days.

Press any key to return to menu...
Enter choice: █
```

(d) Output – Book Issued

```
===== RETURN BOOK =====
Enter Transaction ID: T501
Enter number of days kept: 18
Book returned successfully!
Allowed days      : 14
Days used        : 18
Overdue days     : 4
-----
Book daily fine   : Rs.5.00
Base fine (4 x 5) : Rs.20.00
Member fine multiplier : 1.00
-----
Final fine amount : Rs.20.00
Thank you!
Press any key to return to menu...
Enter choice: █
```

(e) Output – Book Returned & Fine Calculated

```
===== LIBRARY STATISTICS =====
Total Resources      : 25
Registered Members   : 18
Available Resources   : 19
Issued Resources      : 6
Active Borrowings     : 6
Completed Borrowings  : 14
Total Fine Collected : Rs.485.00
-----
Available Percentage  : 76.00 %
Issued Percentage     : 24.00 %
-----
Thank you!
Press any key to return to menu...
Enter choice: █
```

(f) Output – Library Statistics

```
===== DISPLAY BORROWING RECORDS =====
ID      Book ID  Member ID  Issue Day  Due After  Days Used  Status  Fine (Rs.)
-----
T501    B201    M101      01-06-2024 15-06-2024 18         Returned 20.00
T502    B202    M102      02-06-2024 16-06-2024 0          Active   0.00
-----

Press any key to return to menu...
Enter choice: █
```

(g) Output – Borrowing Records

5. Conclusion

The **Smart Public Library Management System** provides a simple and organized way of managing the basic activities of a public library. The system reduces the need for maintaining records manually and makes it easier for the librarian to keep track of books, members, and borrowing transactions.

One of the main purposes of this project was to apply Object-Oriented Programming concepts to a practical problem. The system uses different classes to represent library resources, members, and borrowing transactions. This makes the program easier to understand and also separates different responsibilities within the application.

Inheritance is used to represent different types of library resources. `PrintedMaterial`, `DigitalMaterial`, and `Magazine` are derived from the common `LibraryItem` class. Each derived class provides its own borrowing rules through overridden functions.

The same approach is used for members. `StudentMember`, `Researcher`, and `GeneralMember` inherit from `LibraryMember`. Since each category has different borrowing limits and loan periods, runtime polymorphism allows the system to apply the correct rules automatically.

Encapsulation is another important part of the implementation. The internal data of the classes is protected or private, and operations are performed through member functions. This prevents unnecessary direct modification of the object's data.

References

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition.
2. E. Balagurusamy, *Object Oriented Programming with C++*.
3. Herbert Schildt, *C++: The Complete Reference*.
4. ISO/IEC 14882, *Programming Languages — C++ Standard*.
5. C++ Standard Library documentation and reference material.
6. Course lecture notes on Object-Oriented Programming using C++.
7. GCC documentation and C++ compiler reference material.
8. Online educational resources related to C++ classes, inheritance, polymorphism, STL vectors, and operator overloading.

PIAGIARISM REPORT:

